
ISA 632 – Big Data Analytics and Modern AI

Module 9 – GenAI for Business (2) Evaluation and Deployment

Dr. Jay Shan

jayshan@miamioh.edu



MIAMI UNIVERSITY

Objectives

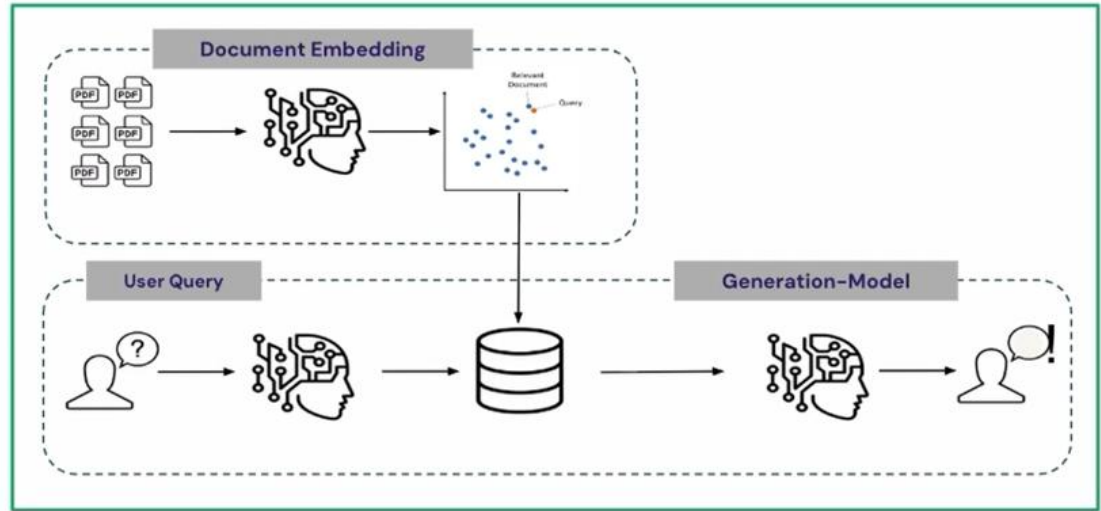
- Describe common evaluation metrics used for evaluating retrieval and generation models.
- Discuss the difference between performance and system metrics.
- Load a model from the model registry for evaluation.
- Identify common built-in evaluation metrics in MLFlow.
- Define a custom evaluation metric with MLflow.
- Run an evaluation test and view the results using the UI or code.
- Deploy a model with Agent Framework.
- Use Review App to interact with the model and collect human feedback.

End-to-End Evaluation

Evaluating the Whole System

➤ What do we evaluate when it comes to the entire AI system?

- Cost metrics
 - Resources
 - Time
- Performance
 - Direct value
 - Indirect value
- Custom metrics
 - for your own use case

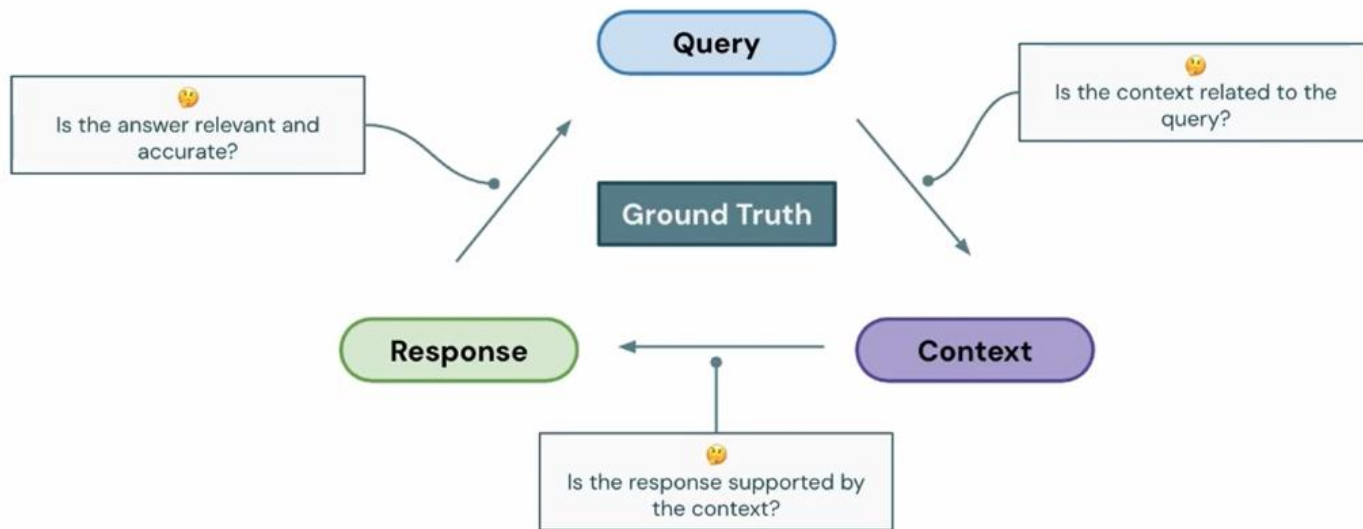


Evaluating RAG Pipeline

- Evaluate each component separately and together.
- Components to evaluate
 - Chunking: method, size
 - Embedding model
 - Vector store
 - Retrieval and re-ranker
 - Generator

Evaluation Metrics

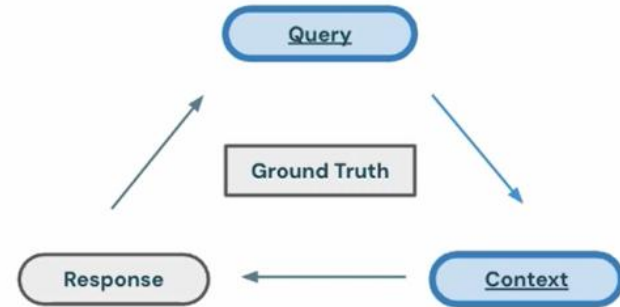
➤ Retrieval and generation related metrics



Retrieval-related Metrics -

➤ Context Precision

- Signal-to-noise ratio for the retrieved context.
- Based on Query and Context(s).
- It assesses whether the chunks/nodes in the retrieval context ranked higher than irrelevant ones.



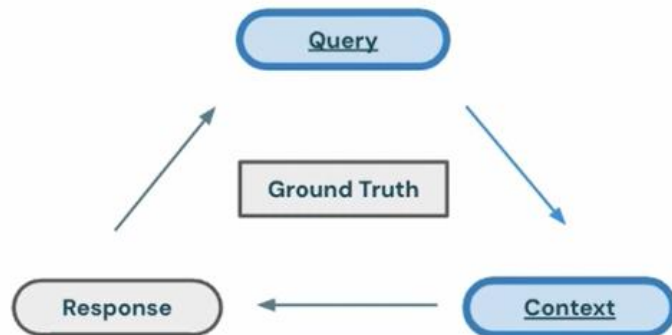
Example:

- **Query:** What was Einstein's role in the development of quantum mechanics?
- **Ground Truth:** Einstein contributed to the development of quantum theory, including his early skepticism and later contributions to quantum mechanics.
- **High Context Precision:** [The contexts specifically mention Einstein's contributions to quantum theory]
- **Low Context Precision:** [The contexts broadly discuss Einstein's life and achievements without specifically addressing his contributions to quantum mechanics.]

Retrieval-related Metrics

➤ Context Relevance

- Measures how relevant the retrieved context is to the query.
- It is evaluated based on the relationship between the query and the retrieved context(s).
- This metric does not assess factual accuracy but focuses on how well the context supports answering the given question.



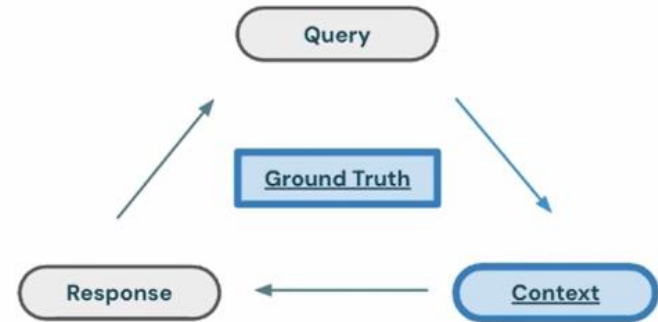
Example:

- **Query:** What was Einstein's role in the development of quantum mechanics?
- **High context relevance:** Einstein initially challenged the quantum theory but later contributed foundational ideas to quantum mechanics.
- **Low context relevance:** Einstein was known for his pacifist views during the early 20th century and became a U.S. citizen in 1940.

Retrieval-related Metrics

➤ Context Recall

- Measures the extent to which all relevant entities and information are retrieved and mentioned in the context provided.
- Based on Ground Truth and retrieved Context(s).



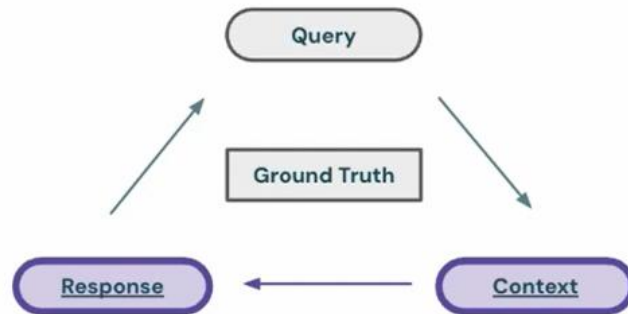
Example:

- **Query:** What significant scientific theories did Einstein contribute to?
- **Ground truth:** Einstein contributed to the theories of relativity and had insights into quantum mechanics.
- **High-recall context:** Einstein contributed to relativity and quantum mechanics.
- **Low-recall context:** Einstein contributed to relativity.

Generation-related Metrics

➤ Faithfulness

- Measures the factual accuracy of the generated answer in relation to the provided context.
- Based on the Response and retrieved Context(s).



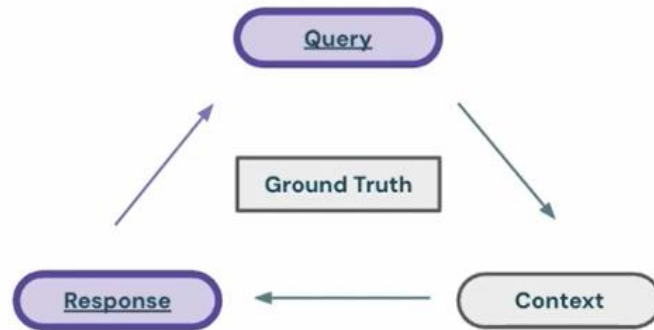
Example:

- **Query:** Where and when was Einstein born?
- **Context:** Albert Einstein (born 14 March 1879) was a German-born theoretical physicist, widely held to be one of the greatest and most influential scientists of all time.
- **High faithfulness answer:** Einstein was born in *Germany on 14th March 1879.*
- **Low faithfulness answer:** Einstein was born in *Germany on 20th March 1879.*

Generation-related Metrics

➤ Answer Relevancy:

- Assesses how pertinent and applicable the generated response is to the user's initial query.
- Based on the alignment of the Response with the user's intent or Query specifics.



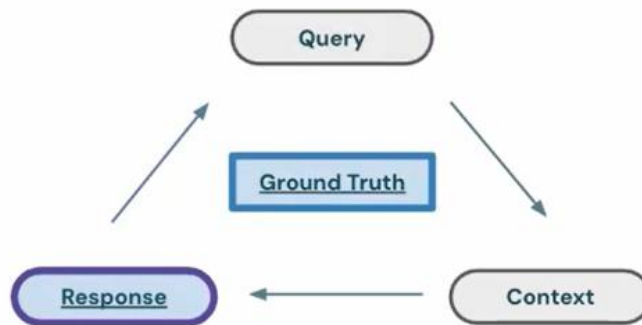
Example:

- **Query:** What is Einstein known for?
- **High relevancy answer:** Einstein is known for developing the theory of relativity.
- **Low relevancy answer:** Einstein was a scientist.

Generation-related Metrics

➤ Answer Correctness

- Measures the accuracy of the generated answer when compared to the ground truth.
- Based on the Ground Truth and the Response.
- Encompasses both semantic and factual similarity with the ground truth.



Example:

- **Ground truth:** Albert Einstein was awarded the Nobel Prize in Physics in 1921 for his explanation of the photoelectric effect.
- **High answer correctness:** Einstein received the Nobel Prize in Physics in 1921 for his work on the photoelectric effect.
- **Low answer correctness:** Einstein won the Nobel Prize in Physics in the 1930s for his theory of relativity.

Custom Metrics for System Evaluation

- What is important to your use case?
 - Do you care about serving latency?
 - Are you concerned with total cost?
 - Are you expecting your system to increase product demand?
 - What about customer satisfaction?
- Custom metrics can be useful for individual components, too.

Custom Metrics in MLflow

- Built-in capabilities for integrating custom metrics into component monitoring
 - Custom Guidelines: Define evaluation rules in natural language for flexible scoring.
 - Unified Scoring: Combine custom and built-in evaluators in one run.
 - Tracked Results: Auto-log scores and aggregates in MLflow experiments.

```
1 # (2) Guidelines scorer for creative_enough (boolean pass/fail)
2 creative_guidelines = Guidelines(
3     name="creative_enough",
4     guidelines="""The response must feel original and imaginative, not a paraphrase of the prompt.
5 It should introduce a concept or angle (e.g., theme, use-case, material pairing) beyond listing features.
6 Keep within ~250 words and avoid tool/planning narration."""
7 )
```

Human Feedback

- Collect data from users and experts
 - Often developers are not the experts of the domain
 - Models' outputs and associated feedback need to be collected and stored in a structured manner
 - Feedback can be explicit or implicit:
 - Explicit feedback: Direct and intentional input from users. Such as ratings, comments and review.
 - Implicit feedback: Gathered indirectly by observing user behavior and interactions.
 - Such as engagement metrics and behavioral data.

Collect Human Feedback with Review App

- Agent Framework provides pre-built feedback app
 - Human feedback is essential for building high-quality GenAI applications that meet user expectations.
 - Agent Framework allows stakeholders to interact with the agent and provide feedback.

The screenshot displays a feedback form titled "In progress" with a "Saved (a few seconds ago)" status. The first question is "Is this a good response for the question?", with three response buttons: "Yes" (thumbs up), "No" (thumbs down), and "I don't know". Below this, a section titled "Tell us why you picked 'Yes'?" contains seven checkboxes: "Accurate", "Relevant", "Safe", "Helpful", "Professional", "Well structured", and "Right length". At the bottom, there is a text input field labeled "Do you have additional feedback?" and a blue "Done" button.

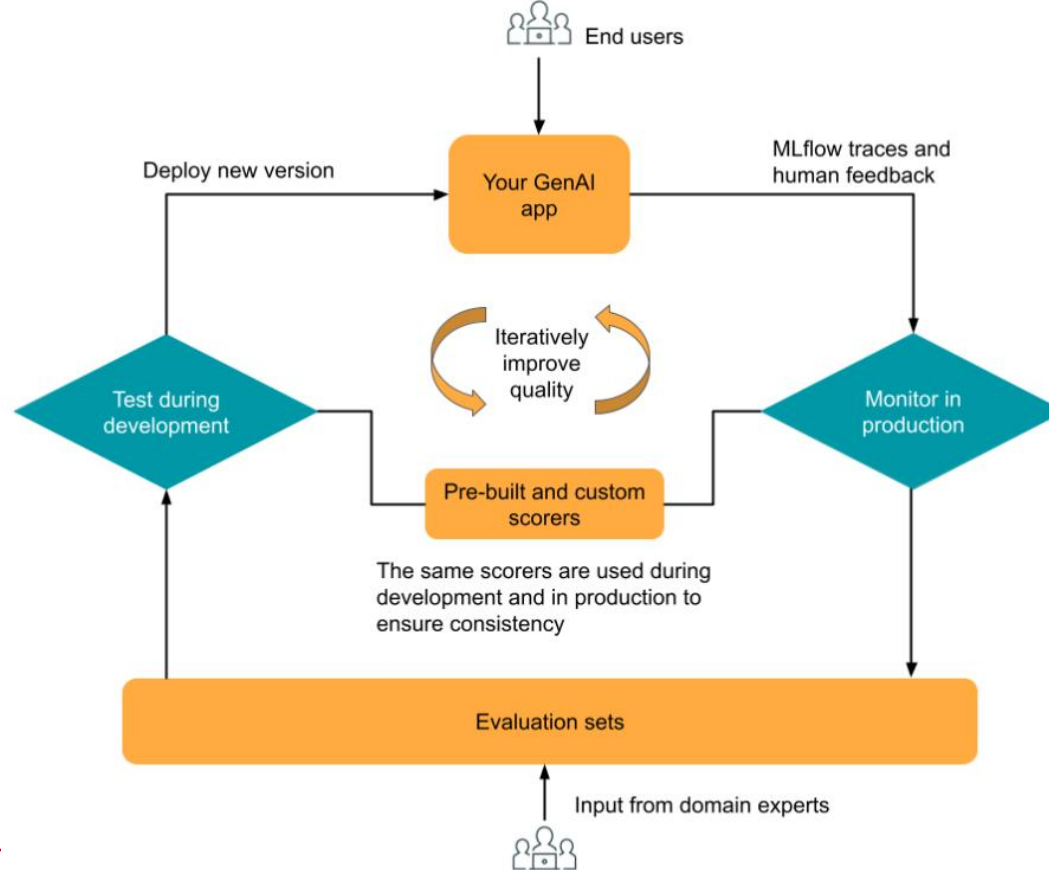
Monitoring Quality in Production

- Automatically run MLflow scorers on traces from your production GenAI apps to continuously monitor quality.
- Use the same scorers in development and production to ensure consistent evaluation.
- Continuous quality assessment with monitoring running in the background.

Lab 3. Evaluation with MLflow 3.0

- Download *isa632-m09_lab3.zip* from Canvas
- Prepare for evaluation
 - Load the model
 - Load the evaluation dataset
- Model Evaluation
 - Build-in metrics
 - Custom metrics
 - Run Evaluation test
- Review the evaluation results

High-level Evaluation and Monitor Workflow



Load the Model

```
import mlflow  
mlflow.set_registry_uri("databricks-uc")
```

```
model_uri =  
f"models:/{DA.catalog_name}.{DA.schema_name}.getstarted_genai_retrieval_demo/1"  
print(model_uri)
```

models:/isa632_7474656346303369.shanz3.getstarted_genai_retrieval_demo/1

Load Evaluation Dataset

display(DA.eval_set)

	inputs	expectations
1	object messages: 0: content: "personalized wooden name sign with app control" role: "user"	> {"creative_enough":true}
2	object messages: 0: content: "handmade lamp with remote dimming" role: "user"	> {"creative_enough":true}
3	object messages: 0: content: "ceramic planter with self-watering insert and moisture indicator" role: "user"	> {"creative_enough":false}

- Sample evaluation sets
 - <https://docs.databricks.com/aws/en/generative-ai/agent-evaluation/evaluation-set#sample-evaluations-sets>

Evaluation Sets

- A good evaluation set has the following characteristics
 - Representative
 - It should accurately reflect the range of requests the application will encounter in production.
 - Challenging
 - It should include difficult and diverse cases to effectively test the full range of the application's capabilities.
 - Continually updated
 - It should be updated regularly to reflect how the application is used and the changing patterns of production traffic.

- Synthesize evaluation sets
 - <https://docs.databricks.com/aws/en/generative-ai/agent-evaluation/synthesize-evaluation-set>

Evaluate(): MLflow 3 vs MLflow 2

	<code>mlflow.genai.evaluate()</code> (MLflow 3)	<code>mlflow.evaluate()</code> (MLflow 2)
Model invocation	You provide a <code>predict_fn</code>	Framework calls the model via model URI
Custom metrics	scorers list	extra_metrics list
Tracing	Native via <code>@mlflow.trace</code>	Limited
Module	<code>mlflow.genai</code>	<code>mlflow</code> (top-level)

Model Evaluation- Preparation

```
import os
import mlflow
from mlflow.genai import evaluate
from mlflow.genai.scorers import Guidelines, RelevanceToQuery, Safety
import mlflow.pyfunc

os.environ["VS_INDEX_NAME"] =
f"{DA.catalog_name}.{DA.schema_name}.product_embeddings"
```


Model Evaluation– Predict Function

(1) Predict fn that calls your registered model

```
model = mlflow.pyfunc.load_model(model_uri)
```

Predict function expected by mlflow.genai.evaluate()

@mlflow.trace

```
def predict_fn(messages):
```

```
    """
```

```
    messages: list of {role, content}
```

```
    Returns: assistant text (str) for scorers.
```

```
    """
```

```
    out = model.predict({"input": messages})
```

Databricks Responses shape: {"object": "response", "output": [{"message...}]}

if isinstance(out, dict) and "output" in out:

```
    try:
```

```
        return out["output"][-1]["content"][0]["text"].strip()
```

```
    except Exception:
```

```
        pass # fall through
```

Older shape: list of message dicts

if isinstance(out, list):

```
    try:
```

```
        return out[-1]["content"][0]["text"].strip()
```

```
    except Exception:
```

```
        pass
```

Fallbacks

if isinstance(out, str):

```
    return out.strip()
```

if isinstance(out, dict):

```
    for k in ("answer", "response", "text", "output"):
```

```
        v = out.get(k)
```

```
        if isinstance(v, str):
```

```
            return v.strip()
```

```
return str(out).strip()
```

Model Evaluation – Guidelines Scorer

(2) Guidelines scorer for creative_enough (boolean pass/fail)

```
creative_guidelines = Guidelines(  
    name="creative_enough",  
    guidelines="""The response must feel original and imaginative, not a paraphrase of the prompt.  
It should introduce a concept or angle (e.g., theme, use-case, material pairing) beyond listing features.  
Keep within ~250 words and avoid tool/planning narration."""  
)
```

Model Evaluation– Run Evaluation

(3) Run evaluation (eval_set has "inputs" and optional "expectations")

```
mlflow.set_experiment(f"/Users/{DA.username}/Retreival_Eval")
```

```
with mlflow.start_run(run_name="eval_guidelines") as eval_run:
```

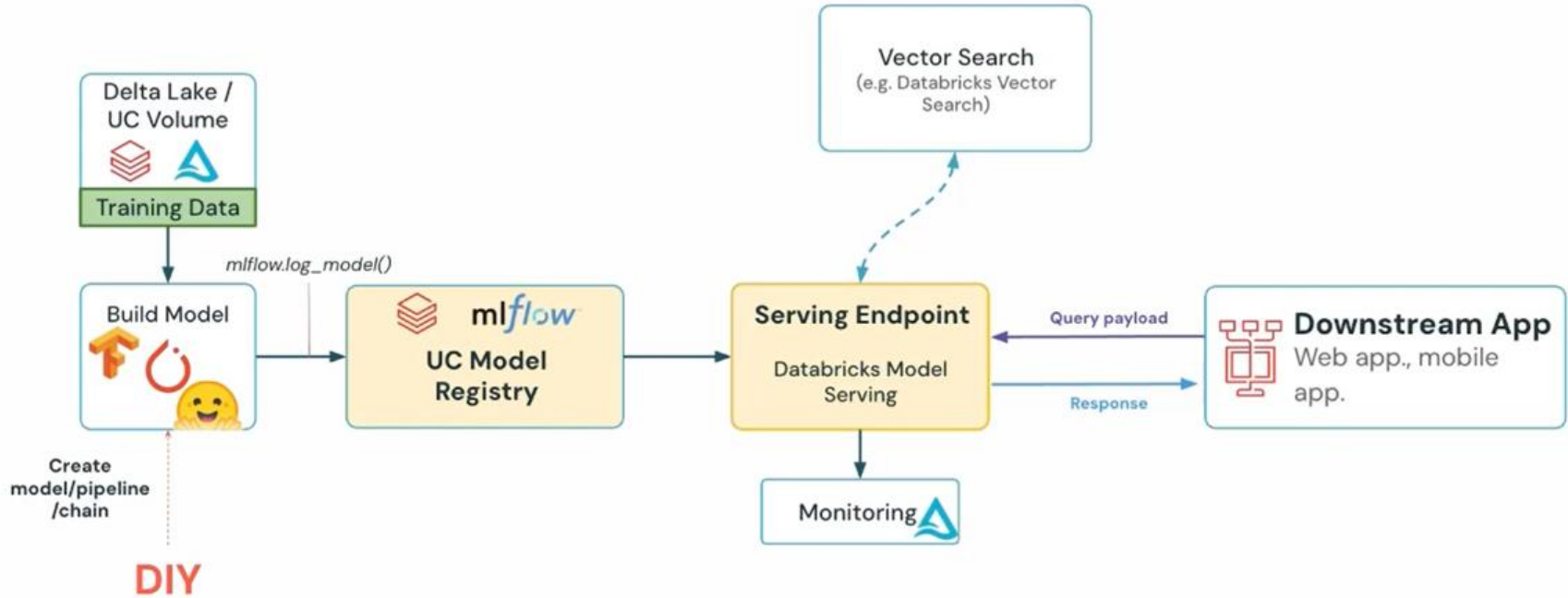
```
    eval_results = evaluate(
        data=DA.eval_set,
        predict_fn=predict_fn,
        scorers=[
            creative_guidelines, # pass/fail vs. our creative_enough label
            RelevanceToQuery(),  # built-in; no ground truth required
            Safety(),            # built-in; no ground truth required
        ]
    )
```

Real-time Deployment with Model Serving

Mosaic AI Model Serving

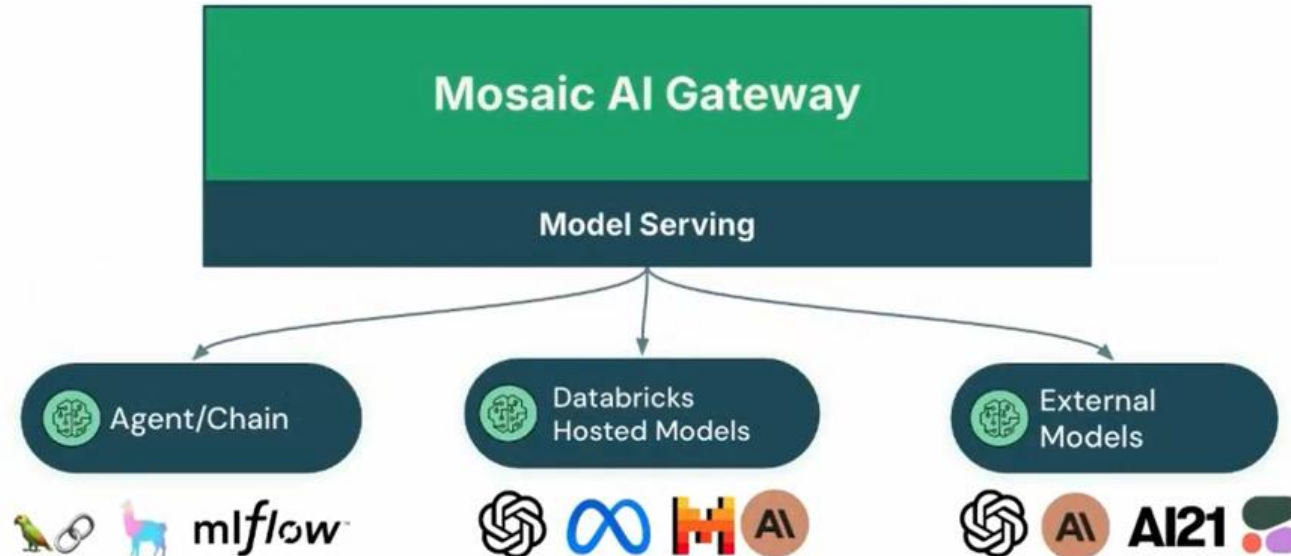
- Production-Grade Serving
 - Highly available, low latency
 - Scalable serving that works for small and large workloads
- Accelerate deployments with Lakehouse-Unified Serving
 - Automatic feature lookups
 - Monitoring and unified governance that automates deployment and reduce errors
- Simplified Deployment
 - Simple and flexible deployment through UI or API

Real-time Deployment



Mosaic AI Gateway

- Protect data and Gen AI deployments through centralized governance



Features

➤ Usage tracking and observability

- Monitor who's using models and how often to help optimize performance and manage costs.

➤ Payload logging

- Capture requests and responses in Unity Catalog, allowing you to debug and improve model quality.

➤ Unified Guardrails

- A single source of truth for data policies, with guardrails like PII detection to ensure compliance.

➤ A/B testing and traffic policies

- Fallback/retry policies.

➤ Permissions and granular rate limiting

- Only authorized users can access models and APIs, while controlling the number of requests to optimize performance.

Lab 4. Real-time Deployment with Model Serving

- Download *isa632-m09_lab3.zip* from Canvas
 - Serve the model with Agent Framework
 - Test the model with Review App
 - Collect human feedback via Review App